Computational Intelligence Semester Report

Ahmet Berk Calisir, s347896

Academic Year 2025-2026

Contents

Chapter 1

Introduction

This report documents the work carried out throughout the course, including Lab 1, Lab 2, Lab 3, and the Final Project. Together, these projects form a progressive exploration of algorithmic problem solving, optimization under constraints, and the design of robust solution strategies for complex computational problems. Each lab introduces a distinct class of techniques and modeling challenges, culminating in a capstone project that integrates and extends the ideas developed earlier in the course.

The laboratory assignments focus on building foundational skills in modeling, algorithm design, and empirical evaluation. Lab 1 emphasizes problem formalization and baseline algorithmic strategies for a multi-dimensional knapsack problem, highlighting the importance of correctness, feasibility, and performance measurement. Lab 2 introduces more advanced optimization techniques and sensitivity to problem structure, encouraging systematic experimentation and comparison of evolutionary and genetic algorithms and operators. Lab 3 further develops these ideas by addressing scalability, algorithmic robustness, and the role of heuristics and approximations in problems where exact methods become impractical. Collectively, the labs establish a methodological framework for approaching complex optimization tasks in a principled and empirical manner.

The Final Project builds directly on this foundation by addressing a non-trivial routing problem with nonlinear, load-dependent costs. Unlike classical routing formulations where traversal costs depend solely on distance or are constrained by explicit capacities, the project problem introduces a continuous penalty that depends jointly on distance traveled and accumulated load. This formulation creates a tight coupling between routing order, trip segmentation, and execution-level cost, requiring solution methods that go beyond purely geometric or greedy heuristics.

From a pedagogical perspective, the project serves as a synthesis exercise. It combines modeling discipline from the early labs, algorithmic experimentation and evaluation practices from the later labs, and additional challenges related to scalability, robustness, and adaptive optimization. In particular, the project emphasizes the importance of selecting appropriate solution representations, separating interacting decision layers, and aligning optimization logic exactly with the problem's cost definition.

This report is structured to reflect my progression throughout the Computational Intelligence course. The laboratory sections present the problem statements, methodologies, and results for each lab in turn, while the project section provides a detailed treatment of the Gold-Collecting Routing Problem, including problem formulation, structural analysis, solution design, experimental evaluation, and discussion. An appendix concludes the report with algorithmic pseudocode and implementation and reproducibility notes for the final project.

Chapter 2

Lab 1: Multi-Dimensional Multi-Knapsack Problem

2.1 Problem Statement and Objective

The first laboratory addresses the *Multi-Dimensional Multi-Knapsack Problem (MD-MKP)*. Given a set of items, each associated with a scalar value and a weight vector across multiple resource dimensions, and a set of identical knapsacks with fixed per-dimension capacities, the task is to assign items to knapsacks so as to:

- respect capacity constraints in every dimension,
- enforce exclusivity (each item assigned to at most one knapsack),
- maximize the total value of selected items.

MD-MKP is NP-hard and becomes rapidly intractable as the number of items, dimensions, and knapsacks increases [6]. The objective of this lab was therefore not to compute exact optima, but to design a **scalable heuristic solver** emphasizing:

1. feasibility at all times,
2. deterministic and reproducible behavior,
3. good solution quality at scale,
4. transparent evaluation and diagnostics.

2.2 Methodology

2.2.1 Design Philosophy

The solver follows a *feasibility-first heuristic paradigm*. All intermediate and final solutions strictly satisfy both capacity and exclusivity constraints, avoiding infeasible states and post-hoc repair. This choice prioritizes robustness and scalability in high-dimensional settings.

2.2.2 Two-Phase Heuristic Solver

The final solver consists of two deterministic phases.

Phase 1: Greedy Initialization Items are ranked using a **constraint-aware normalized scoring function**, inspired by density-based heuristics found in classical knapsack literature [6]:

$$s_i = \frac{v_i}{1 + \sum_d \frac{w_{i,d}}{C_d}},$$

where v_i is the item value, $w_{i,d}$ its weight in dimension d, and C_d the corresponding capacity.

This formulation penalizes items that consume a large fraction of tight resources, unlike mean-weight or density-only heuristics that ignore capacity structure.

Items are processed in descending score order. For each item, a *slack-aware bin score* evaluates compatibility with each knapsack based on remaining capacity, and the item is placed in the best feasible knapsack if possible. If no knapsack can accommodate the item, it is skipped.

This phase produces a fast, fully feasible baseline solution.

Phase 2: Local Improvement Starting from the greedy solution, a local improvement phase attempts to insert previously unassigned items using feasible replacement operators. This aligns with standard local search principles described in evolutionary computing frameworks [3]:

- **Single-item replacement:** insert an item by removing one lower-value item if this yields a positive gain.
- **Two-item kickset replacement:** if necessary, remove up to two low-density items from a restricted candidate set to free sufficient capacity.

The improvement loop terminates when no improving move is found in a full pass or when a fixed iteration limit is reached, guaranteeing bounded runtime and monotonic non-decreasing solution quality.

2.3 Experimental Setup and Results

Three synthetic problem instances of increasing difficulty were evaluated, generated with fixed random seeds to ensure reproducibility:

Problem	Items	Knapsacks	Dimensions
1	20	3	2
2	100	10	10
3	5000	100	100

Key Observations

- **Feasibility:** All greedy and improved solutions satisfied capacity and exclusivity constraints.
- **Phase contribution:** The greedy phase achieved over 99.9% of the final solution value. The improvement phase yielded very small but consistent gains (≈ 0.06–0.07%).
- **Runtime trade-off:** For small and medium instances, improvement overhead was negligible. For the largest instance, runtime increased substantially while gains remained marginal, highlighting a clear quality–time trade-off.
- **Utilization patterns:** Maximum utilization values close to 1.0 indicate tight constraints, while lower mean utilization reflects uneven resource consumption across dimensions.
- **Bound interpretation:** Achieved values exceeding the heuristic reference bound confirm that it is not a true upper bound and must be interpreted cautiously.

2.4 Peer Reviews Received

Two peer reviews were received on Lab 1.

Review from ChristianPanchetti. The reviewer positively evaluated the solver's always-feasible construction, normalized constraint-aware scoring, slack-aware bin selection, and multi-operator local improvement strategy.

The main suggestions concerned improvements in **evaluation transparency and documentation**, including: (i) reporting separate contributions of greedy initialization and local improvement, (ii) explicitly documenting tie-breaking rules and remaining sources of randomness, (iii) stating termination conditions of the improvement phase, (iv) clarifying item selection policies in replacement moves, (v) documenting the capped kickset generation strategy, and (vi) adopting a stronger reference upper bound, as the current heuristic bound can be exceeded.

Overall, the feedback concluded that the algorithmic design is sound and effective, and that addressing these mostly documentation-level refinements would further strengthen presentation rigor.

Review from Gabriele Piovano. This peer review was conducted via a Pull Request that was merged into the main repository. It resulted in targeted refinements focused on correctness checks, reporting clarity, and documentation of performance characteristics.

Based on review annotations in the final code, the contributions included: (i) tightening feasibility validation logic for both capacity and exclusivity constraints, (ii) improving robustness and clarity of the reporting function, (iii) documenting the use of vectorized operations and resulting time complexity, and (iv) explicitly acknowledging limitations of the heuristic upper bound used for evaluation.

The pull request was cleanly merged without conflicts or objections and functioned as effective internal quality assurance.

2.5 Peer Reviews Given

Two detailed peer reviews were provided to classmates, emphasizing constructive and actionable feedback.

Review to gabri1298. Their solution employed a memory-efficient single-array assignment representation, which was acknowledged as a strong engineering choice. The primary critique concerned the use of a mean-weight scoring function, identified as constraint-blind. A normalized, capacity-aware scoring function was proposed (similar to the logic in [6]), with an explanation of its improved handling of bottleneck resources. As an optional enhancement, slack-aware bin assignment was suggested. The review emphasized that existing baselines were already strong and framed the suggestions as theoretical improvements rather than corrections.

Review to ChristianPanchetti. This review addressed a solver based on Simulated Annealing (SA). Two improvement areas were identified: adopting constraint-aware scoring for greedy initialization and expanding reporting diagnostics. The review suggested reporting greedy versus final SA value, utilization metrics, and item counts to quantitatively justify the cost of SA-based search.

2.6 Response to Feedback

The peer reviews highlighted opportunities for improving evaluation transparency and documentation rigor.

While not all suggested refinements were incorporated within the scope of this lab, they are largely orthogonal to the algorithmic core and provide clear directions for future extensions aimed at strengthening experimental rigor and presentation completeness.

2.7 Declarations

Collaboration None.

AI Tools Used ChatGPT and Gemini were used as auxiliary tools for code analysis and debugging, assistance in drafting the README documentation, identifying and reviewing relevant literature, and supporting the analysis and review of peer solutions.

Chapter 3

Lab 2: Traveling Salesman Problem

3.1 Problem Statement and Objective

The second laboratory addresses the *Traveling Salesman Problem (TSP)* and its asymmetric variants (ATSP). Given a set of n cities and a complete cost matrix $D \in \mathbb{R}^{n \times n}$, the objective is to find a Hamiltonian cycle that visits each city exactly once and minimizes total tour cost [8]:

$$\text{Cost}(\tau) = \sum_{i=0}^{n-1} D_{\tau_i, \tau_{(i+1) \bmod n}},$$

where τ is a permutation of the cities.

Three classes of instances were considered:

- **problem_g**: symmetric, metric TSP (triangle inequality holds),
- **problem_r1**: asymmetric, non-metric TSP,
- **problem_r2**: asymmetric TSP with negative edge costs.

The goal of the lab was to design **robust metaheuristic solvers** capable of producing high-quality tours across heterogeneous landscapes, emphasizing correctness, scalability, and interpretability rather than exact optimality.

3.2 Methodology

3.2.1 Representation and Fitness

Solutions are represented as permutations of city indices. Fitness evaluation computes the total tour cost using a fully vectorized formulation, ensuring correctness and efficiency even for large instances. All generated individuals are validated to be proper permutations, guaranteeing feasibility by construction.

3.2.2 Baseline: Hill Climber

A first-improvement Hill Climber using swap moves was implemented as a baseline. This provided a fast, deterministic reference and highlighted the limitations of pure local search, particularly on non-metric and asymmetric instances where local optima dominate.

3.2.3 Genetic Algorithm

The main solver is a Genetic Algorithm (GA) operating on permutations, following standard evolutionary design principles [4]:

- **Tournament selection** ($k = 3$),
- **Order Crossover (OX)** and **Partially Mapped Crossover (PMX)**,
- **Swap** and **Insert** mutation operators,
- **Elitism** to preserve high-quality solutions.

This design supports both conservative exploration (metric instances) and aggressive diversification (asymmetric, non-metric instances).

3.2.4 Self-Adaptive Extensions

Following peer feedback, the solver was refactored into a *self-tuning GA*, utilizing parameter control strategies to handle diverse instance types [2]:

- **Adaptive operator selection**: crossover and mutation operators are chosen probabilistically based on recent success.
- **Adaptive mutation rate**: dynamically adjusted using population diversity (best vs. average fitness).
- **Adaptive elitism**: elitism strength increases during progress and decreases under stagnation.

These mechanisms allow the algorithm to automatically balance exploration and exploitation without manual retuning across instance types.

3.3 Experimental Setup and Results

The official benchmark instances are `problem_*_200`. No fixed evaluation budget was imposed; population size and generations were treated as design parameters and reported transparently.

Key Observations

- **Correctness:** All generated tours were valid permutations.
- **Baseline behavior:** The Hill Climber converged quickly but stagnated early, especially on non-metric instances.
- **GA robustness:** The GA consistently outperformed the baseline across all instance classes.
- **Asymmetry sensitivity:** Inversion-based local search degraded performance on ATSP instances; insertion-based moves proved substantially more effective.
- **Adaptivity gains:** The self-adaptive GA showed improved convergence stability and reduced sensitivity to instance structure.

3.4 Peer Reviews Received

Review from ChristianPanchetti The review highlighted the strong conceptual design of the solver, the correct application of evolutionary operators, and the extensive experimental evaluation. Suggestions focused on structural refinement (reducing redundancy), limiting excessive logging, and strengthening local search operators. The feedback confirmed the correctness of the approach while encouraging further hybridization.

Review from Miki Marconi This review praised the clarity of the implementation, vectorized cost computation, and convergence analysis. The main critique concerned limited operator diversity and the exclusive use of a fixed operator set across heterogeneous instances. The reviewer suggested adaptive or problem-aware operator selection to improve both performance and didactic completeness.

3.5 Peer Reviews Given

Review to ChristianPanchetti The review identified a structural mismatch between 2-opt inversion and asymmetric TSP instances. By benchmarking alternative operators, it was shown that node insertion substantially outperforms inversion on large ATSP instances, yielding improvements exceeding 60% on `problem_r2_1000`. Additional suggestions included improved convergence visualization and reporting.

Review to Miki Marconi The review acknowledged strong greedy initialization and systematic operator comparisons. Constructive feedback focused on making greedy seeding and mutation rates tunable or adaptive to prevent premature convergence on non-metric instances.

3.6 Response to Feedback

Peer feedback directly motivated a substantial refactor of the GA into a self-adaptive solver. Static operator choices were replaced by adaptive mechanisms, addressing concerns about operator mismatch and premature convergence. While not all structural suggestions were incorporated, my final design reflected a clear response to feedback and a significant methodological improvement.

3.7 Declarations

Collaboration None.

AI Tools Used ChatGPT and Gemini were used for code analysis, debugging support, documentation drafting, literature identification, and structured analysis of peer solutions. No core code was directly reused.

Chapter 4

Lab 3: Graph Optimization and All-Pairs Shortest Path

4.1 Problem Statement and Objective

The third laboratory focuses on the *All-Pairs Shortest Path (APSP)* problem in directed weighted graphs. Given a graph with N nodes and weighted edges (possibly including negative weights), the objective is to compute the shortest path cost between every ordered pair of nodes [1].

4.2 Methodology

4.2.1 Problem Setup

Directed graphs were synthetically generated with:

- number of nodes $N \in [10, 1000]$,
- connectivity ranging from sparse (0.2) to dense (1.0),
- edge weights derived from Euclidean distances with optional noise,
- optional negative edge weights, introducing the possibility of negative cycles.

Graphs are represented as dense adjacency matrices, with missing edges encoded as ∞.

4.2.2 Solver Architecture

A centralized `RouteOptimizer` engine dispatches APSP requests to specialized solvers depending on graph properties.

Dijkstra (Non-negative Weights) Implemented via `scipy.sparse.csgraph.shortest_path`. This version leverages CSR sparse matrices and compiled C routines, serving as the correctness reference for non-negative graphs.

Bellman–Ford (Negative Weights) Also executed through SciPy's compiled backend, with explicit detection of negative cycles via `NegativeCycleError`. This allows correct handling of graphs where APSP is undefined.

A* Search Implemented using NetworkX's pure-Python A* algorithm, with a Euclidean heuristic. A critical admissibility issue was identified here: because edge weights are rounded integers, the raw Euclidean heuristic may exceed the true cost, potentially violating admissibility [5].

4.2.3 Key Optimizations

- **Vectorized graph generation:** distance matrices computed using NumPy broadcasting instead of nested loops.
- **Compiled execution:** heavy computation delegated to SciPy's C-based solvers, avoiding Python interpreter overhead.
- **Correctness cross-validation:** results from different algorithms compared on identical inputs to detect subtle bugs.

4.3 Experimental Results and Analysis

Implementation Gap Despite A* being algorithmically more selective, its pure Python implementation is orders of magnitude slower than compiled Dijkstra:

- At $N = 200$: SciPy Dijkstra completes in ~ 0.02s,
- At $N = 200$: NetworkX A* requires ~ 19s.

This demonstrates that implementation details dominate asymptotic advantages in high-level languages.

Complexity Walls Bellman–Ford exhibits its expected $O(N^3)$ behavior. While usable at small scales, runtime explodes beyond $N \approx 500$, making it impractical for dense graphs unless negative weights are strictly required.

Correctness and Negative Cycles For non-negative graphs, all solvers return identical shortest-path costs (within floating-point tolerance). For dense graphs with negative weights, Bellman–Ford frequently reports negative cycles, which is correct behavior rather than a failure of the implementation.

4.4 Peer Reviews Received

Review by Adrien The reviewer praised the use of compiled SciPy solvers, the vectorized graph generation, and the built-in correctness verification. Special appreciation was given to the A* heuristic admissibility fix.

Review by alberto467 The reviewer raised a substantive methodological critique. While the benchmarking framework and performance analysis were considered strong, the solution relied primarily on external libraries rather than on manual implementations of shortest-path algorithms, which are a central learning objective of the course. The reviewer also noted that the A* heuristic leveraged geometric information (node coordinates) that may not be available in abstract graph settings, potentially providing the algorithm with additional structure beyond the problem definition. Overall, the feedback highlighted a misalignment between an engineering-oriented benchmarking focus and the pedagogical expectation of implementing core algorithms explicitly.

Review by AlbeMiglio Through a merged pull request, the reviewer contributed concrete fixes:

- corrected A* heuristic admissibility,
- improved exception handling for negative cycles,

- strengthened correctness checks for finite and infinite paths,
- added dependency specification via `requirements.txt`.

4.5 Peer Reviews Given

One peer review was provided via a confirmed pull request.

Review to Adrien The review analyzed their Python Bellman–Ford implementation and identified severe performance degradation on dense graphs with noise. A compiled drop-in replacement using SciPy was proposed and benchmarked, achieving speedups from $\sim 2\times$ (noise-free) up to over $2500\times$ in noisy settings, while preserving correctness and API compatibility. This review reinforced the central theme of the lab: practical feasibility depends critically on implementation choices.

4.6 Response to Feedback

The peer feedback validated the technical correctness and analytical depth of the work, while highlighting pedagogical limitations related to reliance on external libraries.

Feedback regarding algorithm ownership and heuristic information access is acknowledged as valuable guidance for future extensions, particularly for incorporating manual implementations alongside optimized pre-packaged algorithms.

4.7 Declarations

Collaboration None.

AI Tools Used This lab explicitly made use of AI-assisted code generation and refactoring (ChatGPT and similar tools), particularly for:

- scaffolding experimental code,
- assisting with refactoring and documentation,

All generated code was reviewed, adapted, and validated by the author, and responsibility for correctness remains entirely with the author.

Chapter 5

Final Project: Gold-Collecting Routing Problem

5.1 Problem Definition

We consider a single-vehicle routing problem on a weighted, connected graph, in which a thief must collect gold from a set of cities and transport it back to a central depot. Each city contains a strictly positive amount of gold that must be collected in full when the city is serviced. Partial pickups are not allowed. The vehicle starts and ends its route at the depot, and may return to the depot multiple times during the execution of its route in order to unload collected gold.

The environment is modeled as an undirected graph $G = (V, E)$, where $V = \{0, 1, \ldots, N-1\}$ is the set of cities and 0 denotes the depot. Each edge $(u, v) \in E$ is associated with a positive distance d_{uv}. Each non-depot city $v \in V \setminus \{0\}$ has an associated gold quantity $g_v > 0$, while the depot has $g_0 = 0$. The graph is guaranteed to be connected.

A solution consists of an ordered sequence of visits of the form

$$[(c_1, g_1), (c_2, g_2), \ldots, (c_K, g_K)],$$

where $c_i \in V$ denotes the visited city and $g_i \geq 0$ denotes the amount of gold collected at that visit. The sequence must start and end with $(0, 0)$. A city may be visited multiple times; however, its gold may be collected exactly once and must be collected in full at a single visit. Visits with $g_i = 0$ represent pass-throughs, i.e., traversal of a city without collecting gold.

We maintain a load variable w, representing the total amount of gold currently being carried. The load is initialized to zero at the depot and is reset to zero whenever the depot is visited. When traversing an edge (u, v), the incurred cost depends on both the edge distance and the current load carried by the vehicle.

Formally, the cost of traversing a path P while carrying load w is defined as

$$\text{Cost}(P, w) = \text{Dist}(P) + (\alpha \cdot \text{Dist}(P) \cdot w)^{\beta},$$

where $\text{Dist}(P)$ denotes the total length of the path, and $\alpha > 0$ and $\beta > 0$ are problem parameters controlling the magnitude and curvature of the load-dependent penalty term. This formulation generalizes classical vehicle routing models by introducing a nonlinear, load-sensitive traversal cost, a structure that has been widely studied in routing and transportation literature [11, 7].

The objective is to determine a feasible sequence of visits that collects all gold from every city and minimizes the total accumulated cost over all traversed edges. The problem admits no explicit capacity constraint; instead, the load-dependent penalty implicitly discourages long trips with large carried loads. As a result, the optimal strategy may involve grouping multiple cities into a single trip or performing many short trips with frequent returns to the depot, depending on the problem parameters.

5.1.1 Solution Representation and Feasibility Constraints

A feasible solution to the problem is represented as an ordered sequence of visits, where each visit is encoded as a tuple (v, g_v), with v denoting the visited node and g_v the amount of gold collected at that visit. The sequence must start and end at the depot $(0, 0)$ and may include multiple intermediate returns to the depot, which represent unloading events that reset the carried load to zero.

Cities may be visited multiple times, but gold collection is constrained to occur at most once per city and must be complete: partial pickups are not allowed as they significantly relax the problem constraints. Formally, if a city $v \neq 0$ is visited with $g_v > 0$, then g_v must equal the total gold assigned to that city, and no subsequent visit to the same city may collect additional gold. Visits with $g_v = 0$ are interpreted as pass-through visits and incur no pickup action.

This representation explicitly decouples routing from collection decisions. A solution may traverse a city without collecting gold, which is necessary to ensure feasibility when shortest paths between two target cities pass through intermediate nodes. As a consequence, feasibility cannot be verified solely at the level of city permutations; it must be validated on the expanded visit sequence that includes all intermediate nodes induced by shortest-path traversal, a consideration common in graph-based routing formulations [11].

The feasibility constraints enforced by this representation are: (i) the solution must start and end at the depot; (ii) every city $v \in \{1, \ldots, N-1\}$ must have its gold collected exactly once; (iii) no gold may be collected at the depot; (iv) all consecutive visits (u, v) must correspond to valid edges in the graph. These constraints ensure consistency with the problem definition while allowing flexible routing structures.

The cost of a solution is evaluated edge by edge along the visit sequence. When traversing an edge (u, v), the cost depends on the load carried at that moment, which is determined by the cumulative gold collected since the most recent depot visit. Load accumulation occurs upon leaving a node where gold is collected, and unloading occurs automatically when the depot is visited. This strict edge-by-edge evaluation guarantees exact alignment between the solution representation and the objective function.

5.1.2 Baseline and Lower-Bound Strategy

To contextualize the performance of more advanced solution methods, a simple baseline strategy is defined. The baseline consists of servicing each city independently: for every city $v \neq 0$, the vehicle travels from the depot to v, collects its gold, and immediately returns to the depot before proceeding to the next city. This strategy eliminates all interactions between loads, ensuring that no trip carries gold from more than one city.

Although this approach is suboptimal in many cases, it provides a meaningful reference point. In regimes where the load-dependent penalty is dominant, bundling multiple cities into a single trip can lead to a rapid explosion of cost, making isolated trips surprisingly competitive. Conversely, when the penalty term is weak, the baseline performs poorly because it ignores geometric efficiencies that arise from visiting nearby cities in sequence. Similar singleton-style baselines are commonly used in VRP studies as conservative reference strategies [7].

The baseline does not constitute a strict lower bound on the optimal solution, since it may be outperformed by both geometric and load-aware strategies. However, it serves as a robust normalization reference across different parameter settings. Performance is therefore reported in terms of improvement ratios with respect to the baseline cost, allowing comparisons across instances with widely varying absolute cost scales.

Importantly, the baseline respects all feasibility constraints of the problem and is evaluated using the same edge-by-edge cost function as all other solutions. This ensures that any observed improvement reflects genuine structural advantages of the proposed method rather than artifacts of evaluation or representation.

5.2 Problem Characteristics and Structural Regimes

The problem defined in the previous section generalizes classical routing problems by introducing a nonlinear, load-dependent traversal cost. Unlike the Traveling Salesperson Problem (TSP), where the cost of traversing an edge depends solely on distance, the cost incurred here depends on both distance and the cumulative load carried by the vehicle at that moment. At the same time, unlike classical Capacitated Vehicle Routing Problems (CVRP), no explicit capacity constraint is imposed; instead, carrying larger loads is implicitly discouraged through a penalty term that increases with both distance and load. This places the problem between distance-minimization routing and resource-aware routing formulations studied in the vehicle routing literature [7, 11].

This formulation induces a continuum of problem behaviors that interpolate between geometric routing and resource management. When the load-dependent penalty term is negligible relative to the distance term, the problem effectively reduces to a TSP-like optimization, where the primary objective is to minimize total traveled distance. In contrast, when the penalty term dominates, the problem becomes closer in spirit to knapsack-style or scheduling problems, where the key challenge is deciding which cities to group together in the same trip in order to control the carried load, even at the expense of longer geometric paths. Similar trade-offs between routing efficiency and resource aggregation have been observed in routing problems with nonlinear or state-dependent costs [11].

The relative importance of these two components is governed by the parameters α and β. The parameter α controls the overall scale of the penalty, while β determines its curvature. When $\beta < 1$, the penalty grows sublinearly with respect to the load, favoring longer trips with larger aggregated loads. When $\beta = 1$, the penalty grows linearly, yielding a balanced trade-off between distance and load. When $\beta > 1$, the penalty grows superlinearly, strongly discouraging the accumulation of large loads and incentivizing frequent returns to the depot. Curvature-driven regime changes of this form are well known in optimization problems with nonlinear cost components [3].

Beyond these parametric effects, the structure of the underlying graph also plays a critical role. In sparse graphs, limited connectivity may force the vehicle to traverse long paths between cities, amplifying the impact of load-dependent penalties. In dense graphs, the availability of many short paths may mitigate the geometric cost of frequent depot returns. Consequently, the same parameter settings may induce different effective behaviors depending on graph density and spatial configuration, a phenomenon commonly reported in graph-based routing and network optimization problems [7].

A key difficulty of the problem lies in the interaction between routing and scheduling decisions. The choice of visit order affects not only the traveled distance but also the temporal evolution of the load, which in turn influences the cost of all subsequent edges in the same trip. This coupling breaks the optimal substructure typically exploited in shortest-path and TSP algorithms, making greedy or purely geometric heuristics insufficient in many regimes. Similar breakdowns of optimal substructure have been noted in routing problems with state-dependent costs [11].

As a result, no single optimization strategy is uniformly effective across all parameter settings. Efficient solution methods must adapt their behavior depending on whether the problem is dominated by geometric considerations, load-dependent penalties, or a combination of both. Identifying and exploiting this regime-dependent structure is therefore essential for achieving robust performance across diverse instances, particularly in nonlinear and hybrid routing problems [3].

5.3 Proposed Solution

5.3.1 High-Level Solver Architecture

The proposed solution follows a modular, multi-stage optimization pipeline designed to address both the combinatorial and continuous aspects of the problem. Rather than directly searching over explicit sequences of depot returns and edge traversals, the solver operates on a compact representation and progressively refines candidate solutions through a combination of global exploration and targeted local improvement. This layered design philosophy is common in modern VRP heuristics, where separating decision levels improves scalability and robustness [11].

At a high level, the solver proceeds in four main stages: preprocessing, population-based global search, local refinement, and final solution reconstruction. Each stage operates on a different abstraction level of the problem and serves a distinct purpose within the overall optimization process, reflecting standard decomposition strategies in large-scale combinatorial optimization [7].

In the preprocessing stage, the solver analyzes the input graph and precomputes all-pairs shortest-path distances. This enables efficient evaluation of route segments without explicitly storing intermediate nodes during optimization, a common technique in routing algorithms to reduce evaluation overhead [11]. In parallel, a regime identification step is performed to characterize the instance according to penalty curvature and cost dominance. The resulting regime classification acts as a control signal that influences the behavior of subsequent optimization components.

The core of the solver is a population-based metaheuristic that searches over permutations of cities, representing the order in which gold-collecting cities are serviced. Depot returns are not encoded explicitly at this stage. Instead, for any given permutation, an optimal partition into trips is computed implicitly using a dynamic programming procedure. This separation of concerns allows the solver to explore a reduced search space while still accounting exactly for the

cost impact of depot returns and load accumulation, following the "giant-tour + split" paradigm widely used in VRP heuristics [9].

Global exploration is achieved through a genetic algorithm framework augmented with an island model. Multiple subpopulations evolve in parallel under different strategic priors, encouraging both diversification and intensification. Genetic operators act on city permutations and are combined with adaptive parameter mutation. Periodic migration between islands allows high-quality solutions to propagate while preserving structural diversity, consistent with established results on island-model evolutionary algorithms [3].

To complement the global search, a dedicated refinement phase is applied to high-quality candidate solutions. This phase performs targeted local modifications using exact cost evaluations, focusing on operations that are known to be effective under the identified regime. Refinement aims to exploit structural regularities in the solution that are difficult to discover through purely stochastic operators, a motivation shared with large-neighborhood-based improvement strategies in routing problems [10].

Once optimization terminates, the best permutation is converted into a fully specified, feasible solution. The permutation is split optimally into trips, expanded into an explicit sequence of visits including intermediate nodes along shortest paths, and validated against all feasibility constraints. The final output is evaluated using the same edge-by-edge cost function defined in the problem formulation, ensuring consistency between optimization and reporting.

This architecture deliberately balances generality and specialization. By combining a unified solution representation with regime-aware adaptation and exact cost evaluation at critical stages, the solver achieves robust performance across a wide range of parameter settings while remaining conceptually simple and modular.

5.3.2 Solution Representation

Internally, candidate solutions are represented as permutations of the non-depot cities, i.e., ordered sequences of elements from the set $\{1, \ldots, N-1\}$. This representation specifies the order in which cities are considered for service but does not explicitly encode depot returns or trip boundaries. As a result, the solver operates on a compact search space of size $(N-1)!$, avoiding the combinatorial explosion that would arise from explicitly enumerating all possible return-to-depot decisions. Permutation-based representations are standard in routing metaheuristics due to their simplicity and compatibility with evolutionary operators [4, 11].

Depot returns are deliberately excluded from the representation because their optimal placement depends on the cumulative load carried at different points in the route. Encoding depot visits directly would entangle routing and load-management decisions, significantly increasing the dimensionality of the search space and making genetic operators difficult to design. By postponing the decision of when to return to the depot, the solver can explore city orderings independently of trip segmentation, following a well-established decoupling strategy in VRP heuristics [9].

Given a permutation, the determination of where depot returns should occur is handled by a dedicated optimization procedure rather than by the metaheuristic itself. This separation allows the solver to evaluate the true cost of a permutation exactly, without requiring the genetic algorithm to reason explicitly about load resets or unloading events. As a consequence,

every permutation corresponds to a well-defined optimal set of trips under the problem's cost function.

This representation also enables the solver to support pass-through visits implicitly. During optimization, only pickup cities are ordered; intermediate nodes that arise from shortest-path traversal are not represented explicitly. These nodes are introduced only during solution expansion, after the optimal trip structure has been determined. This design choice ensures that feasibility with respect to graph connectivity is guaranteed while keeping the optimization state minimal, a common practice in shortest-path-based routing solvers [11].

From the perspective of feasibility, the permutation-based representation enforces that each city appears exactly once as a pickup location. Gold collection is therefore structurally constrained to occur exactly once per city, and partial pickups are excluded by construction. The only remaining feasibility conditions—valid edge traversal and correct load evolution—are enforced during route expansion and cost evaluation rather than during permutation manipulation.

Overall, this representation decouples three tightly interacting decisions: city ordering, trip segmentation, and path realization. By allowing each component to be handled at the most appropriate level of abstraction, the solver achieves both computational efficiency and exact alignment with the problem's cost structure. This decoupling is central to enabling regime-aware optimization strategies without sacrificing correctness.

5.3.3 Metaheuristic Optimization

Given the combinatorial complexity of the problem and the strong coupling between routing order and load-dependent costs, exact optimization methods are infeasible beyond very small instances. We therefore adopt a population-based metaheuristic framework centered on a genetic algorithm (GA), a well-established approach for large-scale combinatorial optimization problems [4, 3].

At the core of the solver is a *giant-tour representation*, where each individual encodes a permutation of all non-depot cities. Depot returns are not explicitly represented at this stage; instead, they are introduced optimally by a downstream route-splitting procedure. This decoupling allows the genetic algorithm to focus on optimizing visit order while deferring trip segmentation decisions to an exact dynamic programming routine, consistent with best practices in VRP metaheuristics [11, 9].

To balance exploration and exploitation, the GA is organized as an *island model* composed of multiple semi-isolated populations, each evolving under different strategic priors. In the default configuration, three islands are maintained: an *Exploit* island with low mutation rates and strong local structure preservation, a *Balanced* island using standard genetic parameters, and an *Explore* island characterized by aggressive mutation and structural disruption. Island models are known to improve robustness and prevent premature convergence in complex search spaces [3].

Initialization plays a critical role in accelerating convergence. Rather than relying solely on random permutations, the solver injects a set of high-impact seed solutions generated by diverse constructive heuristics. These include nearest-neighbor and geometric ordering heuristics that emphasize spatial coherence, as well as gold-aware strategies that prioritize cities with large gold quantities. In penalty-dominated regimes, an additional specialized seed is introduced in which cities are ordered by descending gold amount, implicitly favoring isolated or lightly bundled

trips for heavy cities. The use of heterogeneous initial populations is a standard technique for improving early-stage solution quality in evolutionary routing algorithms [4].

Selection is performed using tournament selection, which provides a robust trade-off between selective pressure and population diversity. Recombination between parent solutions is achieved via an ordered crossover operator, which preserves relative ordering information and guarantees feasibility of the resulting permutations. Such operators are widely used in permutation-based genetic algorithms for routing problems [3].

Mutation operators are applied adaptively and consist of three primary actions: swap, inversion (2-opt–style segment reversal), and scramble. The relative frequency of these operators is controlled by island-specific mutation mixes. In exploratory islands, scramble mutations dominate, enabling large-scale structural changes that can escape deep local minima. In exploitative islands, inversion mutations are favored, as they efficiently remove crossings and shorten geometric paths without destroying high-quality subsequences.

Mutation rates and evaluation window sizes are treated as evolvable parameters and are subject to mild stochastic perturbations over time. This self-adaptive mechanism allows the algorithm to automatically adjust its aggressiveness as the search progresses, a strategy commonly employed in modern evolutionary algorithms [3].

To prevent premature convergence, explicit diversity preservation mechanisms are employed. Each island monitors the spread between its best and median individuals; when this spread collapses below a threshold, a controlled *catastrophe* is triggered in which only the elite individual is retained and the remainder of the population is reinitialized randomly. This reset mechanism restores exploratory capacity while preserving the best-known solution.

Periodic migration between islands is performed in a ring topology, allowing high-quality solutions discovered in one strategic context to influence others. Migrants replace the weakest individuals in the target island and are immediately re-evaluated to ensure fitness consistency. This controlled information flow enables cross-fertilization of structural ideas without homogenizing the populations too rapidly.

Finally, because approximate evaluations may be used during evolution (e.g., window-limited route splitting), elite individuals are periodically re-evaluated using the exact cost computation. Any discrepancies are corrected before further selection or migration steps occur. This safeguard ensures that evolutionary pressure is always aligned with the true objective function.

Together, these components form a flexible and robust metaheuristic framework that adapts naturally to the diverse structural regimes induced by the problem parameters. Rather than relying on a single monolithic strategy, the solver leverages heterogeneity, adaptivity, and exact evaluation to achieve consistent performance across distance-dominated, mixed, and penalty-dominated instances.

5.3.4 Local Search and Refinement

While the genetic algorithm is effective at exploring the global structure of the solution space, its operators are inherently stochastic and may leave fine-grained inefficiencies unresolved. To address this, the solver incorporates a dedicated local refinement phase applied to elite solutions produced by the evolutionary process. This hybridization of global evolutionary search with local improvement is a common strategy in high-performing VRP solvers [11].

Refinement is performed using a Large Neighborhood Search (LNS) strategy operating on the permutation representation of the solution. Starting from a high-quality individual, a contiguous subset of cities is removed from the permutation, creating a partial solution. The removed cities are then reinserted one by one using a greedy, cost-aware insertion heuristic. This approach follows the general destroy-and-repair paradigm introduced by Shaw [10].

A key aspect of the refinement procedure is that all candidate insertions are evaluated using the same cost model as the final solution evaluation. Rather than relying on purely geometric proxies, the insertion cost explicitly accounts for the load-dependent penalty term. In penalty-dominated or convex regimes, this load-aware evaluation is critical to avoid seemingly short insertions that would dramatically increase downstream traversal costs due to excessive carried load.

To maintain computational efficiency, refinement operates on a restricted neighborhood and is applied for a limited number of iterations determined by the problem size. This ensures that the refinement phase contributes meaningful improvements without dominating the overall runtime. Importantly, refinement is applied only after the main evolutionary search has converged sufficiently, and its output is accepted only if it yields a strict improvement over the incumbent solution.

Following refinement, the resulting permutation is passed through the exact route-splitting procedure to determine optimal depot return points. This guarantees that any structural improvements introduced during refinement are evaluated under the true cost function and with optimal trip segmentation. As a consequence, the refinement phase cannot introduce infeasible or mis-evaluated solutions.

Empirically, the refinement phase yields modest but consistent gains, particularly in instances characterized by convex penalty growth or mixed cost dominance. In distance-dominated regimes, where geometric structure already dominates, refinement typically has little effect, which is consistent with the near-optimality of the evolved solutions in those cases.

Overall, the combination of global evolutionary search and focused local refinement allows the solver to achieve robust performance across a wide range of parameter settings. The genetic algorithm provides structural diversity and global exploration, while the refinement phase sharpens elite solutions by correcting local inefficiencies that are difficult to eliminate through crossover and mutation alone.

5.4 Experimental Setup

All experiments were conducted on synthetically generated instances produced by the provided problem generator. Each instance consists of an undirected, connected graph with Euclidean edge weights, where node positions are sampled uniformly at random in the unit square. The depot is fixed at the center of the domain, all other cities are assigned strictly positive gold quantities drawn independently from a continuous distribution, and graph connectivity is controlled by a density parameter, with connectivity guaranteed by construction.

Solver performance is evaluated over a grid of configurations varying the number of cities N, the penalty scale α, the penalty curvature β, and graph density. Unless otherwise stated, experiments use $N \in \{20, 30, 40\}$, $\alpha \in \{10^{-3}, 1, 10^{3}\}$, $\beta \in \{0.1, 1, 3\}$, and a fixed density of 0.5. All results are obtained using a fixed random seed, ensuring full reproducibility of instance generation and solver behavior.

For each configuration, the solver is executed once per instance with a computational budget determined as a function of N, controlling population size, number of generations, and refinement iterations. The same budget allocation strategy is used across all regimes to avoid bias. All solution costs—including baseline and optimized solutions—are evaluated using the exact edge-by-edge cost function defined in Section 2, ensuring strict consistency between optimization and evaluation.

Solver performance is assessed relative to a singleton baseline that services each city independently by traveling from the depot to a single city and returning immediately. Results are reported primarily in terms of *improvement ratio*, defined as the ratio between baseline cost and solver cost, allowing meaningful comparison across configurations with widely varying absolute cost scales. We additionally report absolute cost differences, relative percentage improvements, runtime statistics, and structural indicators such as the number of induced trips.

To isolate the contribution of individual solver components, we conduct an ablation study in which key mechanisms—including regime-aware initialization, adaptive mutation, local refinement, and solution-aware regime feedback—are selectively disabled. Each variant is evaluated on the same benchmark grid using identical budgets and random seeds.

All experiments were executed on a single machine under identical conditions, and wall-clock runtime is reported as an indicator of computational cost. The complete experimental pipeline, including instance generation, solver execution, and result aggregation, is fully automated and deterministic.

5.5 Results and Discussion

5.5.1 Overall Performance Trends

Across all tested configurations, improvement over the singleton baseline depends strongly on the curvature parameter β. We observe (i) consistently positive gains in concave regimes ($\beta < 1$), (ii) the smallest (almost non-existing) gains in the linear regime ($\beta = 1$), and (iii) the largest gains when the penalty is strongly convex ($\beta \gg 1$). These trends are summarized in Figure 5.1, which reports the improvement ratio over the singleton baseline across different values of β. Similar curvature-driven effects have been observed in nonlinear and load-sensitive routing variants in the VRP literature [11, 7].

The gains for $\beta \gg 1$ indicate that the baseline, while avoiding multi-city load accumulation, is not generally optimal under a convex penalty. In this regime, the solver can re-order cities and select depot returns so that expensive high-load traversals occur on short shortest-path legs, while avoiding unnecessary detours. This behavior aligns with known observations that even in penalty-heavy routing problems, solution quality can be improved through careful ordering and segmentation rather than purely isolating demands [9].

Conversely, when $\beta = 1$, the regime behaves as a near-balanced setting in which many alternative segmentations and orderings have similar total cost, reducing headroom for optimization and making the baseline comparatively competitive. This flattening of the fitness landscape is a well-known challenge for evolutionary and local-search methods in routing problems with linear cost structures [4, 3].

5.5.2 Concave vs. Convex: Why Density Effects Flip

Graph density interacts with curvature in opposite ways depending on whether the penalty exhibits economies of scale ($\beta < 1$) or diseconomies of scale ($\beta > 1$). For concave penalties, bundling cities is structurally encouraged because marginal penalty growth decreases as load accumulates; in this regime, lower density tends to *increase* improvement over baseline because sparse connectivity makes repeated depot-to-city-to-depot travel especially inefficient, and bundled trips amortize these forced detours. This effect mirrors classical results in routing problems where sparse graphs amplify the inefficiency of naive return-based strategies [7].

For convex penalties, bundling is structurally discouraged because marginal penalty growth increases rapidly with load; here, higher density tends to *increase* improvement over baseline because dense graphs provide many short alternatives, enabling the solver to keep high-load traversals short (or to reset load frequently) without paying excessive geometric overhead. Similar density-dependent trade-offs have been reported in VRP heuristics that combine shortest-path structure with load-sensitive costs [11].

5.5.3 Interpreting Results Through the Regime Lens

The regime identification mechanism provides an explanatory lens for these patterns. The cost-dominance indicator ρ summarizes whether penalty or distance contributions are expected to dominate typical moves, while the curvature β determines whether the solver should prefer consolidation ($\beta < 1$) or aggressive load control ($\beta > 1$). The minimal-gain behavior at $\beta = 1$ is consistent with this interpretation: neither consolidation nor fragmentation is uniformly advantageous, and many solutions concentrate in a narrow cost range. Similar regime-dependent behavior has been noted in evolutionary approaches to routing problems with mixed objectives [3].

We emphasize that ρ is used to steer solver behavior rather than to predict achievable improvement directly; in particular, penalty dominance does not imply that the singleton baseline is optimal, since improvements can still arise from ordering cities to reduce the length of high-load traversals and from exploiting alternative shortest paths enabled by higher density. Consistent with this view, in distance-dominated regimes ($\rho \ll 1$) improvements largely arise from geometric ordering and reducing redundant depot returns, while in penalty-dominated regimes ($\rho \gg 1$) improvements are driven by controlling where load accumulates—e.g., ensuring heavy-load segments are short and selectively grouping only when geometric savings justify the induced penalty.

5.5.4 Effect of Graph Density

Independently of solution quality, density also affects the execution-level structure of the returned solutions. In sparser instances, shortest paths contain more intermediate nodes, increasing the length of the executed action list after expansion and making the edge-by-edge evaluation more sensitive to city ordering and depot reset decisions. This phenomenon is consistent with classical observations on the impact of graph sparsity on routing complexity and solution robustness [11].

5.5.5 Runtime and Scalability

Figure 5.2 illustrates runtime as a function of problem size for different values of β and graph density. Runtime increases with N, reflecting the combined cost of preprocessing (shortest-path computation), population-based optimization, and local refinement; denser graphs and larger instances increase computational load due to larger neighborhoods and more expensive

shortest-path structures. This scaling behavior is consistent with expectations for evolutionary and large-neighborhood-based VRP solvers [10].

Despite this growth, runtimes remain practical for the tested sizes, consistent with the solver's design goal of tractability on medium-sized instances while using exact splitting and evaluation for correctness.

5.5.6 Summary of Findings

Overall, the experimental results show that the proposed solver achieves gains over a strong and feasibility-correct baseline across diverse regimes. The largest improvements occur away from the linear setting, with strong performance both when penalties are concave ($\beta < 1$) and when penalties are strongly convex ($\beta \gg 1$). Furthermore, the interaction between curvature and density is systematic, with sparse graphs amplifying gains in concave regimes and dense graphs amplifying gains in convex regimes. The solver fails to achieve meaningful improvements over the baseline when the number of cities is too small or when the regime is effectively linear, where many competing solutions lie within a narrow cost band.

Bibliography

[1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, MA, 3rd edition, 2009. Referenced for All-Pairs Shortest Path definitions and Bellman-Ford complexity.

[2] Agoston E. Eiben, Robert Hinterding, and Zbigniew Michalewicz. Parameter control in evolutionary algorithms. *IEEE Transactions on Evolutionary Computation*, 3(2):124–141, 1999. Referenced for self-adaptive parameter tuning strategies.

[3] Agoston E. Eiben and James E. Smith. *Introduction to Evolutionary Computing*. Springer, 2nd edition, 2015. Referenced for greedy initialization and adaptive local search principles.

[4] David E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, Reading, MA, 1989. Referenced for standard GA architecture and permutation operators.

[5] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. Referenced for A* admissibility conditions and heuristic properties.

[6] Hans Kellerer, Ulrich Pferschy, and David Pisinger. *Knapsack Problems*. Springer, Berlin, Heidelberg, 2004. Referenced for density-based and pseudo-utility heuristics underlying greedy MKP solvers.

[7] Gilbert Laporte. Fifty years of vehicle routing. *Transportation Science*, 43(4):408–416, 2009.

[8] Eugene L. Lawler, Jan Karel Lenstra, Alexander H. G. Rinnooy Kan, and David B. Shmoys. *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Wiley, Chichester, UK, 1985. Referenced for the standard formulation of symmetric and asymmetric TSP.

[9] Christian Prins. A simple and effective evolutionary algorithm for the vehicle routing problem. *Computers & Operations Research*, 31(12):1985–2002, 2004.

[10] Paul Shaw. Using constraint programming and local search methods to solve vehicle routing problems. In *Principles and Practice of Constraint Programming – CP98*, volume 1520 of *Lecture Notes in Computer Science*, pages 417–431. Springer, 1998.

[11] Paolo Toth and Daniele Vigo. *Vehicle Routing: Problems, Methods, and Applications*. SIAM, Philadelphia, PA, 2nd edition, 2014.

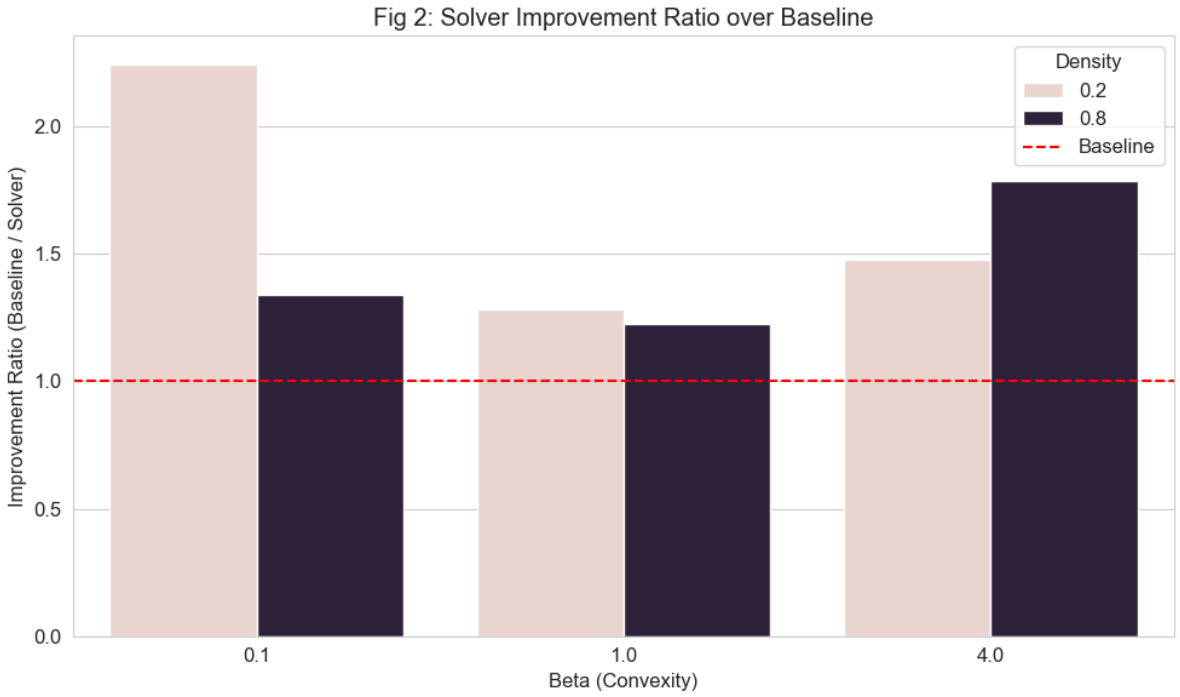

Figure 5.1: Improvement Ratio vs. Baseline

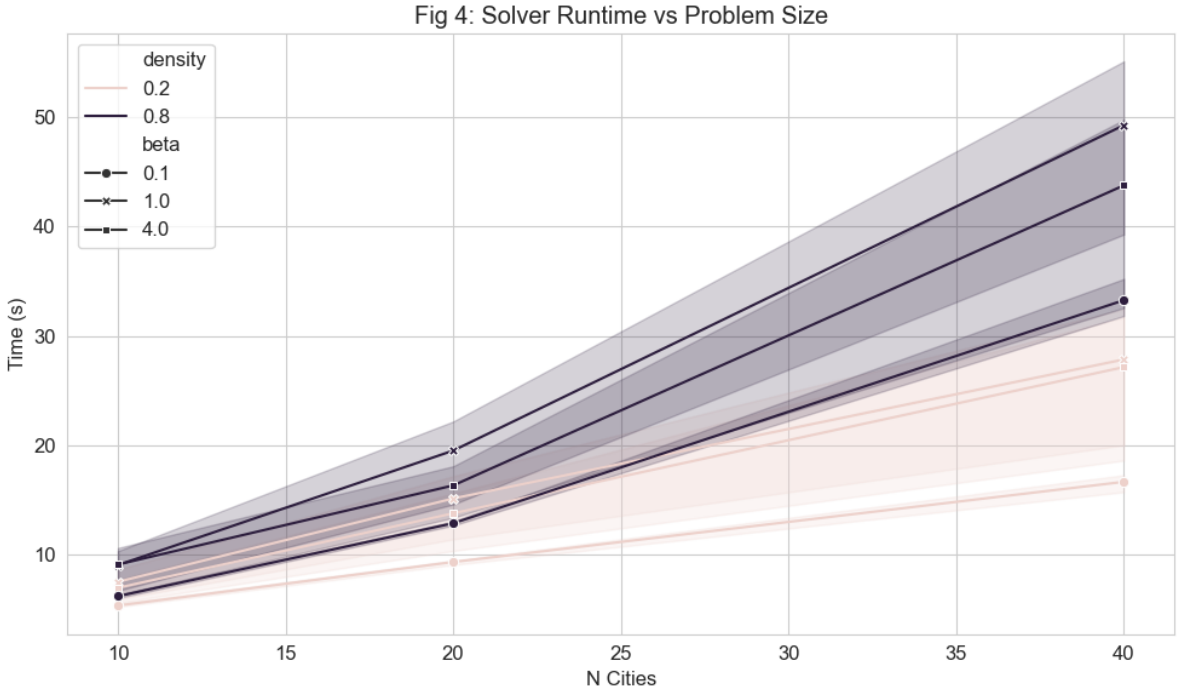

Figure 5.2: Runtime vs. Problem Size

β	Density	N	Baseline	Solver	Imp.	Trips	Time (s)
0.1	0.2	10	32.91	13.40	2.46×	1	5.24
		20	64.90	24.77	2.62×	2	9.08
		40	107.15	48.97	2.19×	4	15.74
	0.8	10	15.77	11.21	1.41×	2	6.04
		20	31.68	21.78	1.45×	3	12.68
		40	64.46	40.75	1.58×	6	31.85
1.0	0.2	10	4 896	4 888	1.00×	4	7.99
		20	7 331	7 319	1.00×	9	16.84
		40	11 092	11 080	1.00×	25	32.49
	0.8	10	1 958	1 958	1.00×	8	10.34
		20	3 968	3 967	1.00×	16	21.78
		40	6 950	6 946	1.00×	32	53.43
4.0	0.2	10	4.1×10^{11}	4.1×10^{11}	1.02×	8	7.72
		20	2.4×10^{11}	1.8×10^{11}	1.32×	16	15.60
		40	2.4×10^{11}	1.9×10^{11}	1.24×	35	31.14
	0.8	10	5.3×10^{10}	5.3×10^{10}	1.00×	9	10.41
		20	1.5×10^{11}	8.8×10^{10}	1.72×	16	18.10
		40	1.6×10^{11}	7.7×10^{10}	2.03×	30	48.96

Table 5.1: Representative benchmark results comparing the proposed solver against the singleton baseline across penalty curvature β, graph density, and problem size N. Improvement is reported as the ratio between baseline and solver cost.

.1 Algorithmic Pseudocode

This appendix documents the core algorithms underlying the proposed solver. The pseudocode reflects the conceptual structure of the Python implementation provided in `solver.py`. Minor implementation-level optimizations, auxiliary bookkeeping, and low-level data handling are omitted for clarity.

.1.1 Overall Solver Pipeline

Algorithm 1 summarizes the high-level optimization pipeline, integrating preprocessing, regime identification, multi-island genetic search, and final local refinement.

Algorithm 1 Overall Solver Pipeline

Require: Graph $G = (V, E)$, parameters α, β, computational budget $B(N)$
Ensure: Feasible route $\mathcal{R}^*$ with cost C^*

Preprocessing:
Compute all-pairs shortest-path distances and auxiliary path-cost structures.
Estimate global cost-dominance indicator ρ_{global} and curvature regime.
Initialization:
Determine population size, number of generations, and refinement budget as a function of N.
Initialize three islands (*Exploit*, *Balanced*, *Explore*) with distinct mutation priors.
Seed populations using geometric, gold-aware, and random heuristics.
for $g \leftarrow 1$ to $G_{\max}$ **do**
    Evolve each island independently (selection, crossover, mutation).
    if $g \bmod 10 = 0$ **then**
        Migrate elite individuals between islands (ring topology).
    end if
    Update global best permutation π_{best}.
end for
Refinement:
$\pi_{\text{refined}} \leftarrow$ LNS-Refinement(π_{best})
Final Evaluation:
$(C^*, \mathcal{T}^*) \leftarrow$ Split-Exact(π_{refined})
$\mathcal{R}^* \leftarrow$ Expand trips $\mathcal{T}^*$ into explicit node sequences.
return $C^*, \mathcal{R}^*$

.1.2 Route Splitting and Exact Cost Evaluation

Algorithm 2 describes the dynamic programming procedure used to optimally partition a city permutation into depot-to-depot trips.

Algorithm 2 Trip Split Procedure

function SPLIT-ROUTE($\pi = (v_1, \ldots, v_n)$, window W)
  Initialize $V[0] = 0$, $V[i] = \infty$ for $i > 0$
  Initialize predecessor array $Pred$
  for $i \leftarrow 0$ to $n - 1$ **do**
    if $V[i] = \infty$ **then continue**
    end if
    $load \leftarrow 0$, $cost \leftarrow 0$, $u \leftarrow 0$
    for $j \leftarrow i + 1$ to $\min(n, i + W)$ **do**
      $v \leftarrow \pi[j]$
      $cost \leftarrow cost + \text{LegCost}(u, v, load)$
      $load \leftarrow load + g_v$
      $total \leftarrow V[i] + cost + \text{LegCost}(v, 0, load)$
      if $total < V[j]$ **then**
        $V[j] \leftarrow total$, $Pred[j] \leftarrow i$
      end if
      $u \leftarrow v$
    end for
  end for
  Reconstruct trips by backtracking $Pred$
  return $V[n]$, trips $\mathcal{T}$
end function

Remark. During evolutionary search, the window size W is bounded for efficiency. All reported final results use an unbounded window, ensuring optimal trip segmentation for the returned solution.

.1.3 Regime Identification

To guide adaptive optimization behavior, the solver estimates a global cost-dominance indicator ρ, measuring the relative influence of load-dependent penalties.

Algorithm 3 Estimation of Cost Dominance ρ

function COMPUTE-RHO-GLOBAL(Sample size K)
  Sample K city pairs (u, v)
  Sample representative load quantiles from gold distribution
  for each (u, v) and load w **do**
    Compute $\ln \rho = \beta(\ln \alpha + \ln w) + \ln S_\beta(u, v) - \ln L(u, v)$
    Store $\ln \rho$
  end for
  $\rho_{\text{global}} \leftarrow \exp(\text{median}(\ln \rho))$
  return ρ_{global}
end function

Logarithmic aggregation improves numerical stability and mitigates the influence of extreme outliers.

.1.4 Local Search via Large Neighborhood Search

Algorithm 4 outlines the local refinement procedure applied to elite solutions using a cost-aware Large Neighborhood Search (LNS) strategy.

Algorithm 4 LNS Refinement with Repair

```
function LNS-Refinement(π, iterations I)
    π_best ← π
    for k ← 1 to I do
        Remove a subset of cities using a randomized destroy operator
        Reinsert cities using greedy cost-aware insertion
        Evaluate candidate via exact split
        if candidate improves cost then
            Update π and π_best
        end if
    end for
    return π_best
end function
```

.2 Implementation and Reproducibility Notes

This section documents implementation choices and experimental protocols to ensure full reproducibility of reported results.

.2.1 Determinism and Randomness

- A single global random seed controls instance generation and solver behavior.
- All stochastic operations rely on `numpy.random.default_rng`.
- For a fixed seed and platform, solver execution is deterministic.

.2.2 Instance Generation

- City coordinates are sampled uniformly from $[0, 1]^2$.
- The depot is fixed at $(0.5, 0.5)$.
- Gold quantities are sampled as $1 + 999 \cdot U(0, 1)$.
- Graph sparsification is controlled by a density parameter; connectivity is enforced by construction.

.2.3 Preprocessing

- All-pairs shortest paths are computed exactly for the tested problem sizes.
- Two matrices are stored:
  - $L[u, v]$: shortest-path distance
  - $S_\beta[u, v]$: sum of edge distances raised to power β

.2.4 Solver Budget and Hyperparameters

Solver budgets scale inversely with N to balance runtime and solution quality.

- Population size: $P = \max(15, \min(50, \lfloor 1200/N \rfloor))$
- Generations: $G_{\max} = \max(15, \lfloor 1500/N \rfloor)$

- LNS iterations: $I_{\text{lns}} = \max(100, \lfloor 15000/N \rfloor)$

All parameters are fixed across experiments.

.2.5 Evaluation Consistency

- All costs are evaluated using the exact edge-by-edge formulation provided by the *Problem* class.
- A verification script confirms numerical agreement between dynamic programming and iterative evaluation.

.2.6 Experimental Pipeline

- Benchmark execution is automated via `benchmark_grid.py`.
- All figures and tables are generated programmatically without manual post-processing.

.2.7 Hardware and Runtime Measurement

- Runtime is measured using `time.perf_counter()`.
- Experiments run on a single-threaded CPU environment.